Decentralized Multi-Robot Collaboration: Scaling Vision-Language-Action Models from Single-Agent Manipulation to Communication- Free Teamwork

Project Workbook

By

Bharat Manivannan (bharat.manivannan@sjsu.edu)
Kanaka Sarat Siripurapu (kanakasarat.siripurapu@sjsu.edu)
Ritwik Reddy Konuganti (ritwikreddy.konuganti@sjsu.edu)
Snehil Raut (snehil.raut@sjsu.edu)

September 29, 2026

Advisor: Dr. Wencen Wu

1.Literature Search, State of the Art

Literature Search

This project fine-tunes a small vision-language-action policy so that one simulated robot arm can see an object and pick it up. The arm is the SO-101. The policy is SmolVLA. The simulator is Isaac Sim 4.5 with Isaac Lab. The task data is one LeRobot dataset, LightwheelAI/leisaac-pick-orange: 60 episodes, 30 frames per second, a front camera and a wrist camera, and a six-number action (shoulder pan, shoulder lift, elbow flex, wrist flex, wrist roll, gripper). The score we will report is task success rate. The first number is the public pretrained checkpoint before any fine-tuning, so later training has a baseline. A second arm, and any test of collaboration without communication, comes after this single-arm setup is running. The literature below is the work that choice sits on.

From web vision-language models to robot actions. Current manipulation policies start from a vision-language model that already knows objects from large-scale image and text training, then add an action head trained on robot trajectories. Brohan et al. (2023) showed the pattern with RT-2: co-fine-tuning a large vision-language model on robot data and web data produces a policy that follows language and generalizes beyond the robot's own training objects. The Open X-Embodiment collaboration (2024) pooled trajectories across many labs and embodiments and trained the RT-X models on that pool. That paper is the dataset-scale argument for generalist policies. Kim et al. (2024) released OpenVLA, an open-weight model of about 7 billion parameters in the same family, so labs without a private stack can fine-tune one. Black et al. (2024) carry the idea into dexterous, high-frequency control with π0, a flow-matching vision-language-action model from Physical Intelligence. These are the leading large policies. They are the wrong size for this semester. A 7-billion-parameter fine-tune does not fit a single-GPU course project, and π0 is not a stack we can install and submit.

Compact policies and community data. Shukor et al. (2025) built SmolVLA for this setting. It has about 450 million parameters: a compact vision-language backbone plus a transformer action expert trained with flow matching (Lipman et al., 2023). Flow matching is the same family of generative models used by π0, and it follows the earlier diffusion policies of Chi et al. (2023), which generate a chunk of future actions instead of a single joint command. SmolVLA is pretrained on community data in the LeRobot format, much of it from low-cost SO-100 and SO-101 arms. The authors report that this small model stays competitive with vision-language-action models about ten times larger, both in simulation and on real SO-100 tasks. Their data guidance matches our set. They recommend on the order of 50 clean episodes of one task, across a few object positions, and they report that 25 episodes were not enough for their pick-and-place study. LightwheelAI/leisaac-pick-orange has 60 episodes of one task, recorded for Isaac, with the two cameras and the six-number action above. The task in that scene is three oranges into a plate followed by a return to rest, so a single episode contains more than one grasp and the score has to be read per orange as well as per episode.

Training is a single-GPU job. The checkpoint and the LeRobot code are public, so the first experiment is to load the pretrained weights, confirm the output is those six numbers, and only then fine-tune. Zhao et al. (2023) are the small-model result underneath this hardware line: ACT showed that imitation learning on a low-cost arm can produce precise manipulation from demonstrations alone. We start from SmolVLA rather than from ACT because the policy has to take images and a language instruction and emit the SO-101 action, which is what the rest of the semester's simulator and dataset are built around.

Simulation has to match the dataset. The six action numbers and the two camera names are a contract. If the simulated arm uses a different joint order, or the scene does not produce `observation.images.front` and `observation.images.wrist`, the policy's outputs are not meaningful. Mittal et al. (2023) built Orbit, the GPU-parallel robot-learning simulator that NVIDIA continued as Isaac Lab on top of Isaac Sim. That is the simulator we are installing. The image is `nvcr.io/nvidia/isaac-sim:4.5.0`. Isaac Sim 5.x needs a newer driver than the 560.35.03 line this install is pinned to, so we stay on 4.5 and on the Isaac Lab release built for that image. The SO-101 arm is loaded from the `leisaac-pick-orange` scene. Todorov, Erez, and Tassa (2012) remain the fallback. MuJoCo is the physics engine in LeRobot's lighter gym environments. If Isaac is not running on a schedule we can keep, the same checkpoint and the same dataset can be evaluated there. The question this semester is whether the arm, the cameras, and the six-number action run as one loop.

Shared weights as a later collaboration idea. Most multi-robot systems coordinate by sending messages, or by giving one central policy every camera. Both assume a link, and both grow with the team. Doshi, Gao, Chen, Finn, and Bohg (2026) test a different setup in CHORUS. One vision-language-action policy is fine-tuned on multi-robot demonstrations. At run time each robot executes its own copy, using only its own cameras and a short identity prompt. Nothing is shared: no camera, no joint state, no messages. On real mobile tasks, including handovers and a two-robot lift, that shared-weight policy beat decentralized policies trained from scratch and also beat centralized baselines. Their backbone, robots, and tasks are not ours. CHORUS is the published example of the project title. This semester does not implement it. This semester finishes the single SO-101 arm, the Isaac scene, and a success rate. Whether a later semester copies the weights onto a second arm, follows CHORUS, or tries a different idea is a decision after that baseline exists.

State-of-the-Art Summary

Leading-edge manipulation is a vision-language-action model: a vision-language backbone trained on large-scale images and text, plus an action head trained on robot trajectories. RT-2 (Brohan et al., 2023) established that pattern. The Open X-Embodiment collaboration (2024) established the multi-lab dataset behind it. OpenVLA (Kim et al., 2024) is the large open-weight version, about 7 billion parameters. π0 (Black et al., 2024) is the industrial flow-matching model aimed at dexterous, high-rate control. Those systems are the performance ceiling. They are also closed, too large for this course, or both.

The leading edge we can run is SmolVLA (Shukor et al., 2025). It is the open model in this family that was built to fine-tune on one GPU and to drive affordable arms, including the SO-101. The authors' data recipe is about 50 clean episodes of one task. Our set, LightwheelAI/leisaac-pick-orange, is 60 episodes of one task, 36,293 frames in total, with a front view, a wrist view, and the six-number action the simulator has to reproduce.LeRobot is the training stack. Isaac Sim 4.5 and Isaac Lab (Mittal et al., 2023) are the simulator. MuJoCo (Todorov et al., 2012) is the fallback if that install slips. ACT (Zhao et al., 2023) and diffusion policy (Chi et al., 2023) are the smaller imitation-learning methods we are not deploying. They do not take a language instruction through a vision-language backbone and emit this arm's action.

CHORUS (Doshi et al., 2026) is the leading published result on collaboration without a message channel: one fine-tuned vision-language-action policy, copied per robot, each robot limited to its own cameras. It is context for the project title. It is not this submission's experiment.

This semester's deliverable is the single-arm setup. A simulated SO-101 is driven by the public SmolVLA checkpoint and then fine-tuned on `LightwheelAI/leisaac-pick-orange`. The reported metric is task success rate in Isaac Sim 4.5. The pretrained checkpoint is scored first, before any training, so the fine-tune has a number to beat. A second arm, and any choice between CHORUS and another collaboration idea, waits until that loop is real.

References

1. Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L. X., Tanner, J., Vuong, Q., Walling, A., Wang, H., & Zhilinsky, U. (2024). π0: A vision-language-action flow model for general robot control. arXiv. https://arxiv.org/abs/2410.24164

Physical Intelligence's industrial vision-language-action model. It uses flow matching and a vision-language backbone to produce high-frequency action chunks for dexterous control. It is the performance reference. We do not train it.

2. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., Florence, P., Fu, C., Arenas, M. G., Gopalakrishnan, K., Han, K., Hausman, K., Herzog, A., Hsu, J., Ichter, B., Irpan, A., Joshi, N., Julian, R., Kalashnikov, D., Kuang, Y., Leal, I., Lee, L., Lee, T.-W. E., Levine, S., Lu, Y., Michalewski, H., Mordatch, I., Pertsch, K., Rao, K., Reymann, K., Ryoo, M., Salazar, G., Sanketi, P., Sermanet, P., Singh, J., Singh, A., Soricut, R., Tran, H., Vanhoucke, V., Vuong, Q., Wahid, A., Welker, S., Wohlhart, P., Wu, J., Xia, F., Xiao, T., Xu, P., Xu, S., Yu, T., & Zitkovich, B. (2023). RT-2: Vision-language-action models transfer web knowledge to robotic control. arXiv. https://arxiv.org/abs/2307.15818

The Google DeepMind paper that turned a vision-language model into a robot policy by co-fine-tuning on web data and robot trajectories. Later vision-language-action models, including the one we fine-tune, follow this pattern at a much smaller size.

3. Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B., & Song, S. (2023). Diffusion policy: Visuomotor policy learning via action diffusion. Robotics: Science and Systems. https://arxiv.org/abs/2303.04137

Peer-reviewed imitation-learning paper. A diffusion process generates a sequence of future actions from camera images. Flow-matching action experts in π0 and SmolVLA are the later version of this idea. We cite the method. We do not train this policy.

4. Doshi, R., Gao, T., Chen, A., Finn, C., & Bohg, J. (2026). CHORUS: Decentralized multi-embodiment collaboration with one VLA policy. arXiv. https://arxiv.org/abs/2606.12352

A single vision-language-action policy is copied onto each robot. Each robot uses only its own cameras and a short identity prompt. No messages pass between robots. This is the published collaboration result behind our project title. This semester's work stops at one SO-101 arm and a success rate.

5. Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., & Finn, C. (2024). OpenVLA: An open-source vision-language-action model. arXiv. https://arxiv.org/abs/2406.09246

The main open vision-language-action model at about 7 billion parameters, from Stanford, Berkeley, and collaborators. It shows what a large open policy can do. It is the size we are not fine-tuning.

6. Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). Flow matching for generative modeling. International Conference on Learning Representations. https://arxiv.org/abs/2210.02747

Peer-reviewed generative-modeling paper. SmolVLA's action expert is trained with this family of models, which is why a forward pass returns a chunk of joint targets rather than one unconstrained label.

7. Mittal, M., Yu, C., Yu, Q., Liu, J., Rudin, N., Hoeller, D., Yuan, J. L., Singh, R., Guo, Y., Mazhar, H., Mandlekar, A., Babich, B., State, G., Hutter, M., & Garg, A. (2023). Orbit: A unified simulation framework for interactive robot learning environments. IEEE Robotics and Automation Letters, 8(6), 3740–3747. https://arxiv.org/abs/2301.04195

Peer-reviewed paper behind Isaac Lab. It is a GPU-parallel simulator for robot learning, and NVIDIA continued it as Isaac Lab on Isaac Sim. We install Isaac Sim 4.5.0 and the Isaac Lab release matched to that image, then load the SO-101 scene from `leisaac-pick-orange`.

8. Open X-Embodiment Collaboration. (2024). Open X-Embodiment: Robotic learning datasets and RT-X models. IEEE International Conference on Robotics and Automation. https://arxiv.org/abs/2310.08864

Peer-reviewed multi-lab dataset and model paper. It is the scale argument for generalist robot policies. Our fine-tune uses 60 episodes of one pick, not this corpus.

9. Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., Alibert, S., Cord, M., Wolf, T., & Cadene, R. (2025). SmolVLA: A vision-language-action model for affordable and efficient robotics. arXiv. https://arxiv.org/abs/2506.01844

Our policy. About 450 million parameters, pretrained on LeRobot community data from low-cost arms, with a public checkpoint and a single-GPU fine-tune. The authors' 50-episode guidance is why the training set is `LightwheelAI/leisaac-pick-orange` and not a large multi-task dump. The first test is one dataset frame in, six joint numbers out.

10. Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. IEEE/RSJ International Conference on Intelligent Robots and Systems, 5026–5033.

Peer-reviewed physics engine. It is the simulator inside LeRobot's gym environments. If Isaac Sim 4.5 does not stay on schedule, evaluation moves here. The checkpoint, the dataset, and the success rate stay the same.

11. Zhao, T. Z., Kumar, V., Levine, S., & Finn, C. (2023). Learning fine-grained bimanual manipulation with low-cost hardware. Robotics: Science and Systems. https://arxiv.org/abs/2304.13705

Peer-reviewed ACT paper. Imitation learning on a low-cost arm, using action chunks from demonstrations. It is the small-model result for this class of hardware. SmolVLA is the policy we load because it also takes a language instruction and was trained in the same data format as our SO-101 episodes.

2.Project Justification

Large vision-language-action models can already follow an instruction and move a robot arm. RT-2, OpenVLA, and π0 set that result. They also sit out of reach of a normal lab: billions of parameters, private data, or a training stack that cannot be installed and graded. The open model that was built for the other end of that range is SmolVLA, about 450 million parameters, pretrained on community data from low-cost SO-100 and SO-101 arms, with a public checkpoint and a single-GPU fine-tune. Its authors state that about 50 clean episodes of one task are enough to start, and that 25 were not enough. What that paper does not hand a later project is a closed-loop score of the public checkpoint inside Isaac Sim, on a public SO-101 dataset that was recorded for Isaac, with the same six joint actions and the same front and wrist cameras the scene has to render. Without that number, a claim about a second arm, or about collaboration with no messages, has nothing to beat.

That missing baseline is this project. We load the pretrained SmolVLA checkpoint and check that one frame of `LightwheelAI/leisaac-pick-orange` produces the SO-101 action: shoulder pan, shoulder lift, elbow flex, wrist flex, wrist roll, and gripper. We run that arm in Isaac Sim 4.5 and the matching Isaac Lab release, in the scene that dataset was recorded in, and we confirm the arm moves and both cameras render. We then score the untouched checkpoint on the pick task. A success rate of zero is an acceptable first result. It means the evaluation script, the scene, and the action format agree. We fine-tune on this 60-episode set on an HPC H100 or A100, then score the fine-tuned policy with the same script. The deliverable is the pair of success rates, the checkpoint, and a setup another group can rerun. The dataset, the weights, and the simulator image are all public. Isaac Sim is pinned to 4.5.0 because Isaac 5.x does not run on the driver this install targets. Training and simulation are already assigned to machines we have: the campus H100 and A100 queues for fine-tuning, and an RTX 4090 workstation for Isaac.

This is the kind of result that holds up at Expo and that a workshop paper can be written from. Generalist policies are not reproducible here. A fully specified SO-101 baseline is. Anyone who later asks whether two copies of the same weights can pick, hand over, or lift together without a communication channel needs exactly this measurement first. CHORUS is the published example of that later question: one shared vision-language-action policy, each robot limited to its own cameras, no messages. We do not run that experiment this semester. We build the single-arm score it would be compared against, on a model and a robot small enough to finish, with the zero-training rate and the fine-tuned rate reported the same way. A sponsor or a reviewer can see the gap, the method, and the number. That is the justification.

3.Identify Baseline Approaches

The baseline is an existing system we run unchanged, then compare against our own fine-tune using the same robot, the same scene, and the same score.

Baseline we will run:

The baseline is the public pretrained SmolVLA checkpoint, with no fine-tuning (Shukor et al., 2025).

Item	Choice
Method	SmolVLA, about 450 million parameters, vision-language backbone plus a flow-matching action expert
Package	LeRobot, https://github.com/huggingface/lerobot
Weights	lerobot/smolvla_base on the Hugging Face Hub
Dataset the checkpoint is scored on	LightwheelAI/leisaac-pick-orange, 60 episodes, 30 fps
Robot	SO-101 in that dataset's Isaac scene
Simulator	Isaac Sim nvcr.io/nvidia/isaac-sim:4.5.0 and the Isaac Lab release built for Isaac Sim 4.5. Isaac Sim 5 is not the baseline.
Cameras	observation.images.front and observation.images.wrist, 480×640 color
Action	Six numbers, in this order: shoulder pan, shoulder lift, elbow flex, wrist flex, wrist roll, gripper
Language	The pick instruction that accompanies this dataset, passed through LeRobot's SmolVLA preprocessor
Training on this baseline	None. Weights stay at the public checkpoint.

LeRobot already implements loading, preprocessing, and inference. Our code calls that path. A separate check, before any simulator episode, runs one dataset frame through the checkpoint and must print those six numbers. If the vector length or the order is different, that checkpoint is not a valid baseline for this arm, and we stop.

The simulator side of the baseline is also an existing scene, not a robot we build. Isaac Lab loads the SO-101 from the `leisaac-pick-orange` scene. A rollout counts only if three things are true: the arm moves, the command is the same six-number vector, and both cameras render. The RTX 4090 workstation is the simulation machine. Fine-tuning, later, is a separate step on an HPC H100 or A100. The baseline itself does not need that job.

The evaluation harness already exists. The script is LeIsaac's `policy_inference.py`, and SmolVLA is served to it through LeRobot's async policy server as `--policy_type=lerobot-smolvla`. We are not writing the loop, we are connecting our checkpoint to it.

How the baseline is scored

One Python evaluation script drives Isaac Lab, queries the pretrained checkpoint, and prints a success rate. The same script is the only scorer for every later checkpoint.

- One trial is one episode in the `leisaac-pick-orange` scene, 120 seconds, both cameras on.
- The task is three oranges onto the plate. We score per-orange success: oranges placed, divided by three times the number of episodes. We also report the per-episode count, from 0 to 3.
- Success is detected by the environment's own check, using the `LeIsaac-SO101-PickOrange-Direct-v0` variant that exposes it, not a rule we invent.
- The first baseline pools at least three runs of 5 rounds, with the scene seeds written down in the script, so the fine-tuned policy can be run on the same seeds. A single short run is noise and is not reported as a result.
- A success rate of 0% on the pretrained checkpoint is a completed baseline. The script ran, the arm was the SO-101, and the action format matched. That number is what fine-tuning has to beat.

The dataset notes still have to confirm that "holding the orange clear of the table" matches the task those 60 episodes actually demonstrate. If the episodes use a different goal, the script's success test changes to that goal before any number is reported. The trial count, the seeds, and the six-number action order do not change.

How we pin the stack

Isaac Sim is pinned to 4.5.0, because Isaac 5.x does not run on the driver line this install targets. One LeRobot version is used for both the baseline and the fine-tune and recorded in the lab log, because on this exact task changing only the LeRobot version moved a published checkpoint from 44% to 27% per-orange success, with the same data and seed. If the baseline and the fine-tune ran on different versions, part of the difference between them would be the framework, and we could not separate the two.

What we compare to the baseline

The comparison system is the same SmolVLA model, fine-tuned with LeRobot on `LightwheelAI/leisaac-pick-orange` only, then loaded into the same evaluation script. Same scene, same cameras, same action order, same 20 seeds, same success test. The result of the project is the two rates side by side: pretrained checkpoint, then fine-tuned checkpoint.

A short run on 3 to 5 episodes is a training smoke test inside LeRobot. It is not a baseline and it is not the comparison system. Those weights are not scored as the project result.

Related approaches we will not run

These are the other state-of-the-art choices from Chapter 1. Each has an implementation. None of them is our baseline.

Approach	Implementation	Why it is not the baseline
OpenVLA (Kim et al., 2024)	https://github.com/openvla/openvla	Open and runnable, and about 7 billion parameters. It is not trained for this SO-101 action, and a fine-tune of that size is outside this semester.
π0 (Black et al., 2024)	Physical Intelligence openpi	Industrial flow-matching vision-language-action model. Different weights, different robot set, and not the checkpoint our dataset loader is built for.

ACT (Zhao et al., 2023)	Included in LeRobot	Real open-source imitation-learning baseline for low-cost arms. It does not take a language instruction through the SmolVLA backbone, so a win or a loss against ACT would be a different project.
Diffusion Policy (Chi et al., 2023)	Included in LeRobot	Same reason as ACT. It is the earlier action-chunk method, and it is not the public checkpoint we are scoring.
CHORUS (Doshi et al., 2026)	Paper and project site. No SO-101 Isaac package we can install as the baseline.	Two or more robots, no communication channel. This semester has one arm. CHORUS needs a single-arm success rate before it can be compared with anything. That rate is the baseline defined above.

MuJoCo through LeRobot's gym environments stays a fallback simulator if Isaac Sim 4.5 is not running. It is not a second baseline. If evaluation moves there, the checkpoint, the dataset, the six-number action, the 20 seeds, and the success test stay as written here.

4.Dependencies and Deliverables

Dependencies

These are the dependencies that can halt the project, not the full package list. Each one, if it fails, stops a specific piece of work, so each is paired with what we already have as a fallback.Deliverables

- An RT-core GPU for the simulator. Isaac Sim renders the cameras our policy needs, and that requires ray-tracing cores. The campus H100 and A100 queues do not have them, so they cannot run Isaac at all. The simulator runs on Dr. Wu's RTX 4090 workstation. If that machine is unavailable, the fallback is a cloud RTX instance, and past that, MuJoCo through LeRobot's gym environments. Without one of these three, there is no closed loop and no success rate.

- The Isaac Sim 4.5 / driver pairing. The install is pinned to Isaac Sim 4.5.0 because Isaac 5.x does not run on the driver line this project targets (560.35.03 on the login-node A40). Isaac Lab and LeIsaac each have to match that 4.5 image. A mismatched pair fails at import. If no LeIsaac release works with Isaac Sim 4.5 on the workstation's driver, that is the first thing we escalate, because the pin protects the rest of the stack.

- A single, fixed LeRobot version. One LeRobot version is used for both the baseline and the fine-tune. On this exact task, changing only the version moved a published checkpoint from 44% to 27% per-orange success. If the baseline and the fine-tune run on different versions, the difference between our two numbers is partly the framework and the comparison is void.

- HPC training access. The fine-tune is roughly 20,000 steps, about four hours on one A100, on the campus H100 or A100 queues. If that access lapses, training stops. Inference and the baseline do not depend on it.

- The public assets: SmolVLA checkpoint and the dataset. The baseline is the pretrained `lerobot/smolvla_base` checkpoint, and the fine-tune trains on `LightwheelAI/leisaac-pick-orange`. Both are public on the Hugging Face Hub and Apache-2.0. If either were pulled, we already hold local copies; the project does not depend on them staying online.

Deliverables

These are the project's own deliverables. Course paperwork is not listed here.

- **A reproducible evaluation harness.** One Python script that loads a SmolVLA checkpoint, drives the SO-101 in the `leisaac-pick-orange` Isaac scene through LeIsaac's `policy_inference.py`, and prints a per-orange success rate under a fixed protocol (seeds, pooled runs, both cameras). This is the core artifact. Every checkpoint is scored by it.

- **A zero-training baseline score.** The pretrained checkpoint's per-orange success rate on the pick-orange task, with per-episode logs committed. A rate of 0% is an acceptable, complete result: it proves the loop runs.

- **A fine-tuned SmolVLA checkpoint and its score.** The model fine-tuned on the 60-episode set on HPC, scored by the same harness on the same seeds. The headline result is the two rates side by side, pretrained and fine-tuned.

- T**he pinned environment recipe.** Driver, Isaac Sim version, Isaac Lab version, LeIsaac version, and LeRobot version, recorded so another person can rebuild the stack and rerun a scored episode.

- **Rollout videos and a short failure catalog.** Saved successes and failures from the evaluation runs, with a brief note on how the arm fails. This is the raw material for the report and for semester 2.

- **A written report and a reproducible README.** The methods and results, and instructions precise enough that a teammate can clone the repository, run one command, and reproduce a training run and an evaluation.

- **A derivative paper,** if the results support it. Dr. Wu has framed a fully specified SO-101 baseline as potentially publishable. A short workshop-style writeup of the baseline and fine-tune numbers is a stretch deliverable, conditional on the results, not a committed one.

5.Project Architecture

The system is deliberately split across two machines, because the two jobs have different hardware needs. Training a policy needs any CUDA GPU. Rendering the cameras the policy reads needs a GPU with ray-tracing cores. No single machine we have does both well, so the architecture separates them and moves one file between them: the checkpoint.

The two halves

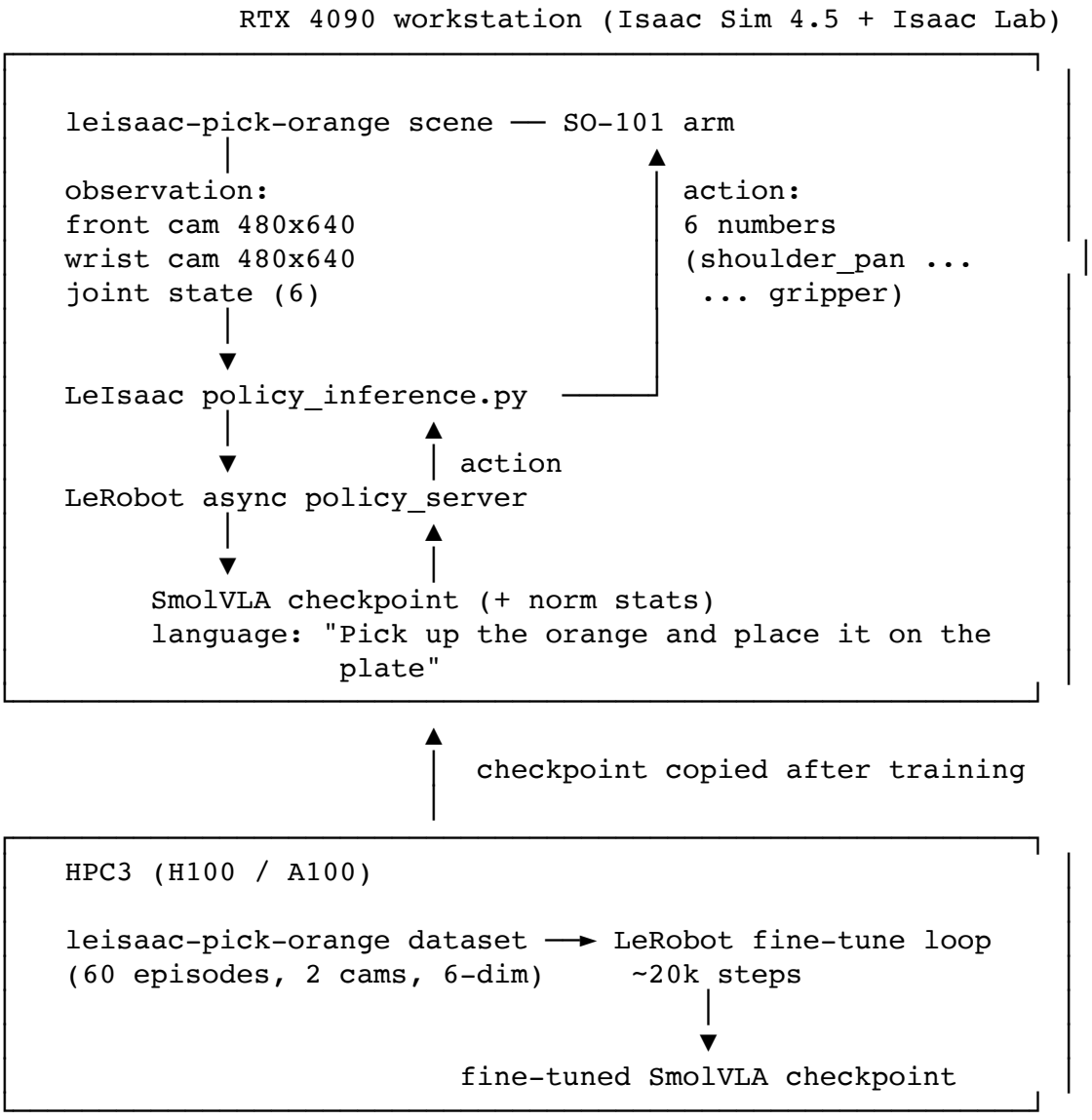
Training side (HPC3, H100 or A100). This is where SmolVLA is fine-tuned. It reads the `leisaac-pick-orange` dataset (60 episodes, two camera streams, six-number actions) and runs the LeRobot training loop for about 20,000 steps. Its only output is a checkpoint plus its normalization statistics. It never renders anything and never needs ray-tracing cores.

Simulation side (RTX 4090 workstation). This is where a checkpoint is scored. Isaac Sim 4.5 with Isaac Lab loads the SO-101 into the pick-orange scene. At each step the scene emits the two camera images and the arm's joint state; the policy returns a six-number action; the arm executes it. LeIsaac's `policy_inference.py` runs this loop, and LeRobot's async policy server holds the checkpoint and answers with actions. This side produces the success rate.

What crosses between them

Only the checkpoint. After a training run on HPC, the fine-tuned weights and their normalization statistics are copied to the workstation and loaded by the same evaluation harness that scored the pretrained baseline. Nothing else is shared: no live connection, no dataset streaming between machines. This keeps the comparison honest, because the pretrained checkpoint and the fine-tuned checkpoint are scored by identical code on identical seeds.

The data flow inside one evaluation loop



Why this shape

Three properties fall out of it, and each maps to a requirement from the earlier chapters.

- *The action and camera contract is the interface*. The scene must emit `observation.images.front` and `observation.images.wrist` at 480×640, and the policy must return six numbers in the dataset's joint order. That contract is checked before any episode runs, with a single-frame test that needs no simulator. If it fails, nothing downstream is meaningful.
- *The scorer is fixed*. One evaluation script scores every checkpoint, so the pretrained baseline and every fine-tune are directly comparable. The success signal comes from the environment's own check, not from a rule we add on top.
- *The simulator is swappable, the rest is not*. If Isaac Sim 4.5 cannot be stood up on schedule, the simulation half moves to MuJoCo through LeRobot's gym environments. The checkpoint, the dataset, the action format, and the scoring protocol are unchanged. The architecture isolates that risk to one box in the diagram.

A note on scope and semester 2. This architecture serves one arm. The semester 2 idea, one shared policy copied onto two arms with no communication, is a change to the simulation side only: a second SO-101 in the scene, each running its own copy of the same checkpoint on its own cameras. The training side and the checkpoint interface do not change. The single-arm architecture here is what that extension builds on, which is why the interface contract and the fixed scorer matter now.

6.Evaluation Methodology

Everything here is scored by one evaluation script on one environment, so that a pretrained checkpoint, a fine-tuned checkpoint, and any external reference are directly comparable. Chapter 3 identified the baselines; this chapter is how they are measured.

Testbed setup

The evaluation runs on the RTX 4090 workstation. The stack is Isaac Sim 4.5.0, the matching Isaac Lab release, and LeIsaac for the SO-101 pick-orange scene. The policy is served by LeRobot's async policy server and reached by LeIsaac's `scripts/evaluation/policy_inference.py` over a local host and port. Training, when it happens, is on the HPC3 H100 or A100 queue and is not part of the testbed; only the resulting checkpoint crosses to the workstation.

Every reported run records the environment it ran under: driver version, Isaac Sim version, Isaac Lab version, LeIsaac version, and the single LeRobot version used for both training and evaluation. This is not bookkeeping. A published checkpoint on this exact task moved from 44% to 27% per-orange success on a LeRobot version change alone, so the version is part of the result.

Datasets used in evaluation

`LightwheelAI/leisaac-pick-orange`: 60 episodes, 30 fps, robot type `so101_follower`, six-float action and state (`shoulder_pan.pos`, `shoulder_lift.pos`, `elbow_flex.pos`, `wrist_flex.pos`, `wrist_roll.pos`, `gripper.pos`), two camera streams `observation.images.front` and `observation.images.wrist` at 480×640. This dataset defines the scene the policy is evaluated in and the action-and-camera contract the policy must satisfy. Evaluation is closed-loop in the simulator, not scored against held-out dataset frames; the dataset supplies the task, the embodiment, and later the fine-tuning data.

Baseline approaches

From Chapter 3, run in this order:

- B0, random action. Sample the action space each step. The floor. Proves a later non-zero score means something.
- B1, SmolVLA zero-shot. `lerobot/smolvla_base`, no fine-tuning, served as `--policy_type=lerobot-smolvla`. The primary baseline. A score near zero is expected and acceptable, because the checkpoint has never seen this scene.
- B2, published same-task checkpoints (external, quoted not rerun). The `wsagi/ACT-PickOrange` checkpoint reports 44.0% per-orange (33/75, 95% CI

[29.5%, 58.5%]) on Isaac Sim 5.1; `shadowHokage/act_policy` reports 18.3%. These are indicative context, not our numbers, and they ran on a different simulator build.

- B3, SmolVLA fine-tuned on the 60-episode set, scored by the same script (Workbook 2 / later).

Test parameters

Fixed now so all runs are comparable:

- Task `LeIsaac-SO101-PickOrange-v0`, or the `-Direct-v0` variant where the programmatic success check is needed.
- `--episode_length_s=120`, `--step_hz=30`, cameras enabled, `--device=cuda`.
- Language instruction held constant: `"Pick up the orange and place it on the plate"`.
- `--eval_rounds=5`, at least three independent runs pooled. Single-run spread on this task is about ±40%, so one round is noise and is not reported.
- Seeds recorded per run and reused across checkpoints.
- `--policy_action_horizon` swept per checkpoint, and the chosen value published. The default of 16 is not assumed; the reference work found the best horizon was 70 for one checkpoint and 32 for another.

Performance measures

- Per-orange success rate. Oranges placed on the plate divided by three times the number of episodes. The headline number, comparable to the published table.
- Per-episode count. Oranges placed per episode, 0 to 3, reported as a histogram. This is required, not optional: the observed distribution on this task is bimodal (episodes cluster at 0 and 3), so a mean alone hides the behavior.
- 95% confidence interval on the per-orange rate, with numerator and denominator shown.
- Success is read from the environment's own check, not a criterion we invent.

Reward, when B4's reinforcement-learning extension is reached, is a training signal only and is never reported as a performance measure. Success rate is the only number compared across rows.

Experiments to execute (Workbook 2)

1. Single-frame contract test: one `leisaac-pick-orange` frame through `lerobot/smolvla_base`, printing six numbers in the dataset's joint order. No simulator. Pass/fail gate for everything after.
2. B0 random-action rollout under the fixed protocol; record per-episode logs.
3. B1 zero-shot SmolVLA under the fixed protocol; record per-episode logs.
4. Horizon sweep for `smolvla_base`; publish the chosen value.

Results (to be filled after execution)

Run	Policy	Per-orange success	Per-episode mean	95% CI	Horizon	Sim version
B0	Random action	—	—	—	n/a	Isaac 4.5.0
B1	SmolVLA zero-shot	—	—	—	swept	Isaac 4.5.0
B2	ACT (external)	44.0% (33/75)	1.32	[29.5%, 58.5%]	70	Isaac 5.1
B3	SmolVLA fine-tuned	—	—	—	swept	Isaac 4.5.0

A 0% in the B1 row is a completed baseline. The milestone is that the row can be filled at all: the scene, the action format, the cameras, and the scorer agree end to end.

7.System Design/Methodology

Solution Overview

The solution is a closed loop between a policy and a simulated robot, plus an offline training step that produces the policy. It reuses three existing systems (Isaac Lab, LeIsaac, LeRobot) and adds one component of our own: a thin evaluation wrapper that runs many seeded episodes and logs per-episode outcomes. The design goal is that a single checkpoint file is the only thing that changes between the baseline and the fine-tuned result, so any change in the score is attributable to training and nothing else.

Components

- Scene and robot (LeIsaac + Isaac Lab). Provides `LeIsaac-SO101-PickOrange-v0`: the SO-101 arm, the pick-orange table scene, the two cameras, and the programmatic success check (via the `-Direct-v0` variant). We do not modify it.
- Policy server (LeRobot). `lerobot.async_inference.policy_server` holds a checkpoint and its normalization statistics and answers observation-to-action requests. Swapping the checkpoint path swaps the policy without touching anything else.
- Inference client (LeIsaac). `scripts/evaluation/policy_inference.py` steps the scene, sends observations to the policy server, and applies the returned actions. `--policy_type=lerobot-smolvla` is the path our model uses.
- Evaluation wrapper (ours). Runs the client for N rounds across M seeds, writes one row per episode (oranges placed, termination reason, seed, horizon, checkpoint id, simulator version), and prints the pooled per-orange rate with a confidence interval. This is the only code we write.
- Trainer (LeRobot, on HPC). Fine-tunes SmolVLA on the 60-episode dataset and emits a checkpoint. Not part of the runtime loop.

Data and control flow (pseudo-code)

The evaluation loop:

```
[loop goes here, will be ready by next assignment]
```

Algorithmic details that affect performance

- Action horizon. SmolVLA and the ACT reference predict a chunk of future actions; the client re-queries every `--policy_action_horizon` steps. The best horizon differs per checkpoint (70 and 32 were the tuned values for two ACT checkpoints; the default is 16). We sweep it and publish the chosen value, because leaving it at the default silently costs accuracy.
- Normalization statistics. The checkpoint's input/output normalization must travel with the weights. A checkpoint scored with the wrong statistics produces confident garbage, so the wrapper logs which statistics were loaded.
- Version pinning. One LeRobot version for training and evaluation. This is an algorithmic concern here, not just packaging: a dataloader and loss change across versions moved a same-task checkpoint from 44% to 27%.

Deployment / usage (tutorial)

[steps go here, will be ready by next assignment]

8.Implementation Plan and Progress

The implementation is broken into stages that match the two-machine architecture: environment first, then the offline model checks, then the simulator, then the closed loop, then training. The plan mirrors the team workflow. Progress is tracked in the table at the end and updated as each item lands.

Plan

Stage 0 — Environment and hardware. Confirm HPC3 training access. Secure a ray-tracing GPU for the simulator. Pull the Isaac Sim 4.5.0 container. Pin the driver/Isaac pairing.

Stage 1 — Offline model checks (no simulator). Install LeRobot at the pinned version. Load `lerobot/smolvla_base`. Run one dataset frame through it and confirm the output is six numbers in the SO-101 joint order. This is the contract test and it gates everything after.

Stage 2 — Simulator stand-up. Install Isaac Lab (matched to 4.5) and LeIsaac. Run a stock example unmodified. Load the SO-101 into the `leisaac-pick-orange` scene and confirm it is visible.

Stage 3 — One rollout. Drive the arm from a policy through LeIsaac's `policy_inference.py`. Confirm three things: the arm moves, the action is six numbers, both cameras render.

Stage 4 — Close the loop and score. Wire the evaluation wrapper around the inference script. Run B0 (random) and B1 (zero-shot SmolVLA) under the fixed protocol and print a pooled per-orange success rate. A 0% for B1 is a pass.

Stage 5 — Training. Overfit 3–5 episodes on HPC as a smoke test (proves training runs). Then the full fine-tune (~20k steps). Score the fine-tuned checkpoint with the same wrapper and seeds.

Hard gate. If the simulator (Stage 2) is not running on schedule, the simulation side moves to MuJoCo through LeRobot's gym environments. The checkpoint, dataset, action format, and scoring protocol do not change. This gate is timeboxed so October is not spent fighting an installer.

Progress

Item	Stage	Status	Notes
HPC3 access, GPU job runs end to end	0	Done	A100 job ran and exited cleanly
Ray-tracing GPU secured for the simulator	0	Done	Dr. Wu's RTX 4090 workstation; RunPod A40 used as an interim while access was pending
Isaac Sim 4.5.0 container pulled	0	Done	Pulled and rendering on the login-node A40; pinned to 4.5 for driver 560.35.03
Driver / Isaac version pinning decided	0	Done	Isaac 5.x excluded on this driver
LeRobot installed at a pinned version	1	Pending	Blocked on choosing the single train+eval version
SmolVLA loads; one frame → six numbers	1	Pending	Contract test; needs no simulator, can run before Stage 2
Isaac Lab + LeIsaac installed on the 4090	2	Pending	First thing to verify: LeIsaac/Isaac-Lab 4.5 pairing imports
Stock Isaac Lab example runs	2	Pending	
SO-101 loaded and visible in pick-orange scene	2	Pending	
One rollout: arm moves, 6-dim action, cameras render	3	Pending	
Evaluation wrapper (seeded runs, per-episode logs)	4	Pending	The one component we write

B0 random-action baseline scored	4	Pending	
B1 zero-shot SmolVLA baseline scored	4	Pending	0% is a pass
Overfit smoke test (3–5 episodes) on HPC	5	Pending	Not started until Stages 1–4 land
Full fine-tune (~20k steps) + scored	5	Pending	

What is proven so far. The training machine works and the Isaac Sim 4.5 image renders on an RT-core GPU, so the two hard hardware unknowns from the start of the project are closed. The remaining work is software integration: the LeRobot version pin, the contract test, and the LeIsaac-on-4.5 pairing, in that order. None of those needs new hardware.

Reproduction. Each completed item is recorded in the lab log with the exact command and the environment versions, so the progress table above is backed by a runnable trail and not just checkmarks.

9.Project Schedule

The schedule is organized into five two-week sprints from late September to the December 4 advisor demonstration. Each teammate owns one lane end to end, so tasks do not overlap. Class assignments are not tasks here; a few external dates are shown only as milestone anchors.

Team lanes (owner → responsibility, whole semester)

- Ritwik — Simulator and integration. RunPod/4090 setup, Isaac Sim 4.5, LeIsaac scene, the closed loop, and HPC fine-tuning runs.
- Snehil — Model and inference. SmolVLA loading, the single-frame contract test, and the SmolVLA-over-LeRobot policy-server path.
- Sarat — Evaluation and data. The evaluation wrapper, the metric definitions, the dataset notes, and the scored baselines.
- Bharat — Research and documentation. Reference-paper stack, report chapters, meeting logs, and reproduction README.

Sprints

Sprint 1 (Sep 29 – Oct 11): Environment and contract

Owner	Task
Ritwik	Stand up the 4090 workstation; install Isaac Sim 4.5 + Isaac Lab + LeIsaac; run a stock example
Snehil	Install LeRobot at the pinned version; load smolvla_base; one fram
Sarat, Bharat	Write dataset notes for leisaac-pick-orange; draft the success definition and metric
Bharat	Assemble the reference stack from the team PPT; set up the meeting log

Milestone: contract test passes; environment recorded in lab log.

Sprint 2 (Oct 12 – Oct 25): Loop and baselines

Owner	Task
Ritwik	Load SO-101 in the pick-orange scene; one rollout (arm moves, 6-dim action, cameras render)
[Teammate B]	Bring up the SmolVLA policy-server path; sweep the action horizon
[Teammate C]	Build the evaluation wrapper; score B0 (random) and B1 (zero-shot SmolVLA)
[Teammate D]	Draft report Chapters 1–2; keep version pins documented

Milestone: B0 and B1 per-orange rates logged (0% for B1 is a pass).

Sprint 3 (Oct 26 – Nov 8): Fine-tune

Owner	Task
Ritwik	Overfit smoke test (3–5 episodes) on HPC; then full fine-tune (~20k steps)
[Teammate B]	Sync fine-tuned checkpoint + norm stats to the 4090; verify it loads
[Teammate C]	Score B3 (fine-tuned) on the same seeds; produce the baseline-vs-fine-tune table
[Teammate D]	Write up methods/results so far; failure-video labeling begins

Milestone: B3 scored; pretrained vs fine-tuned rates side by side.

Sprint 4 (Nov 9 – Nov 22): RL fine-tune and iteration

Owner	Task
Ritwik	B4 residual RL from the B3 warm start (conditional on Sprint 3 landing)
[Teammate B]	Support the RL reward/observation wiring; re-run horizon sweep on new checkpoints
[Teammate C]	Score B4; lock the eval protocol (seeds, pooled runs, CI)
[Teammate D]	Build the failure catalog; draft the results narrative

Milestone: B4 numbers in hand (or documented as slipped, per the gate).

Sprint 5 (Nov 23 – Dec 4): Freeze, present, demo

Owner	Task
Ritwik	Freeze and tag the baseline checkpoint; prepare the live demo
[Teammate B]	Verify one-command reproduction of a scored run on a clean setup
[Teammate C]	Final baseline table with confidence intervals; commit all per-episode logs
[Teammate D]	Finish reproduction README and report chapters; end-of-November results presentation

Milestone: results presented (end of Nov); prototype demonstrated to advisor (Dec 4).

Key project milestones

Date	Milestone
Oct 11	Contract test passes; simulator go/no-go gate (else switch to MuJoCo)
Oct 25	Zero-training baseline scored (B0, B1)
Oct 31	Full fine-tune complete and scored (B3)
Nov 22	RL fine-tune results (B4)
End of Nov	Results presentation
Dec 4	Early prototype demonstrated to advisor

External anchors (not project tasks): Workbook 1 window closes early October; Workbook 2 window closes end of October; report first chapters and the advisor demo fall on December 4. The sprints are placed so project work lands ahead of these, not on top of them.